Coggins' Search Method

Samuel Dark

May 8, 2026

Introduction

Introduction i

- Search methods for unconstrained optimization fall into two broad categories [2]:
  1. **Direct search methods**: require only objective function evaluations; no derivatives are needed.
  2. **Descent methods**: exploit gradient information to determine search directions.
- The Coggins search method is a direct optimization technique that iteratively searches for an optimum by adaptively adjusting step sizes.
- It is an interpolation-based approach that requires few function evaluations and uses unequally spaced points to bracket the minimum, followed by successive quadratic approximations resulting in rapid convergence [1].

Coggins' Search Mehtod

Algorithm i

The classical Coggins method is a combination of the single-variable search technique proposed by Davies et al. and Powell [1]. It is designed to optimize an objective function of a *single* variable $x \in \mathbb{R}$.

The algorithm proceeds as follows [2]:

1. A starting point x_0 is chosen and the objective function $f(x_0)$ is evaluated.
2. The variable is incremented by a step size Δx and $f(x_0 + \Delta x)$ is evaluated.
    - If a function improvement is obtained, the step size is **doubled** for the next evaluation.

Algorithm ii

- If no improvement is obtained on the first step, the direction is **reversed** and the next point is located at a distance Δx from the starting point.

3. This bracketing continues until a local optimum is straddled, yielding three points around the optimum $(x_{k-2},\ x_{k-1},\ x_k)$.
4. An additional point x_{k+1} is then located at:

$$x_{k+1} = x_{k-1} + \frac{\Delta x}{2} \tag{1}$$

where Δx is the current step size. The best three points are retained, say (x_1, x_2, x_3).

Algorithm iii

5. The **quadratic interpolation** is used to iteratively to find the optimum. The optimum x^* is located by setting $\frac{df}{dx} = 0$, giving:

$$x^* = \frac{(x_2^2 - x_3^2)f(x_1) + (x_3^2 - x_1^2)f(x_2) + (x_1^2 - x_2^2)f(x_3)}{2\big[(x_2 - x_3)f(x_1) + (x_3 - x_1)f(x_2) + (x_1 - x_2)f(x_3)\big]} \tag{2}$$

6. The objective function at x^* is compared with the best previous point subject to a convergence criterion:

$$|f(x^*) - f(x_{\text{best}})| \leq \varepsilon \tag{3}$$

Algorithm iv

7. If the criterion is satisfied, the procedure stops. Otherwise, the worst point is replaced by x^*, a new quadratic surface is fitted, and steps 4–6 are repeated.

Algorithm Implementation

Algorithm Implementation

The Coggins method was implemented using Python. The implementation follows three stages:

1. **Define** $f(x)$
2. **Bracketing Phase**: locate three points straddling the minimum using step-doubling and $x_{k+1} = x_b + \frac{\Delta x}{2}$
3. **Quadratic Fitting**: repeatedly fit a quadratic to the three best points and compute x^* until $|f(x^*_{\text{new}}) - f(x^*_{\text{best}})| \leq \varepsilon_{\text{lim}}$

Stage 1: Objective Function

We define the test function to be minimized:

$$f(x) = x^4 - 14x^3 + 60x^2 - 70x, \qquad x_0 = 0, \quad \Delta x = 1$$

where x_0 is our intial point and Δx is the step-size.

```
1 def f(x):
2     return x**4 - 14*x**3 + 60*x**2 - 70*x
```

This function has two local minima. Cogins' method will find the nearest minimum.

Stage 2: Bracketing and Step-Doubling i

- Starting at $x_0 = 0$, $f(x_0)$ is evaluated.

```
    def bracket_minimum(func, x0, delta=1.0):
    xa = x0
    fa = func(xa)
```

- We step forward and evaluate $f(x_0 + \Delta x)$.

```
    xb = xa + delta
    fb = func(xb)
```

Stage 2: Bracketing and Step-Doubling ii

- Upon improvement, we continuously doubled the step size and evaluated the function at each new point until $f(x_k) \geq f(x_{k-1})$, confirming the minimum is straddled.

  The loop ends when the bracketed points (x_{k-2}, x_{k-1}, x_k) satisfy the condition; $f(x_{k-2}) \geq f(x_{k-1})$ and $f(x_k) \geq f(x_{k-1})$

- After confirming the straddle, we locate an additional point $x_{k+1} = x_{k-1} + \Delta x/2$ to improve accuracy

```
1          x_k1 = xb + delta / 2.0
2      f_k1 = func(x_k1)
```

Stage 2: Bracketing and Step-Doubling iii

The best three out points out of the four is bracketed.

```
Bracketing from x0 = 0.0,  initial delta = 1.0

Point                    x           f(x)  Action

xa                0.000000       0.000000  starting poi
xb                1.000000     -23.000000  first step
xc (iter 1)       3.000000      33.000000  delta double

Straddle confirmed: f(xc) >= f(xb).
Minimum lies between xa = 0.0000 and xc = 3.0000.
```

Stage 2: Bracketing and Step-Doubling iv

```
x_(k+1)                2.000000        4.000000  fourth point

Best three points retained (sorted by x):

x1                     0.000000        0.000000
x2                     1.000000      -23.000000  <- lowest f
x3                     2.000000        4.000000

Straddle check:  f(x1) >= f(x2): True,  f(x3) >= f(x2): T
```

The best three points now tightly straddle the optimum and are ready for quadratic fitting

Stage 3: Quadratic fitting

With x_1, x_2, x_3 straddling the optimum, we compute x^* [2]

```
1       def quadratic_minimum(x1, x2, x3, f1, f2, f3):
2       numerator = (
3           (x2**2 - x3**2) * f1 +
4           (x3**2 - x1**2) * f2 +
5           (x1**2 - x2**2) * f3
6       )
7       denominator = 2.0 * (
8           (x2 - x3) * f1 +
9           (x3 - x1) * f2 +
10          (x1 - x2) * f3
11      )
12
13      return numerator / denominator
```

We iterate until $|f(x^*_{\text{new}}) - f(x^*_{\text{best}})| \leq \varepsilon$: where we chose $\varepsilon = 10^{-6}$

Quadratic Fitting (output)

```
Quadratic Refinement

  Iter             x*           f(x*)            |f|

     1       0.960000      -23.440957       4.41e-01
     2       0.824477      -24.311850       8.71e-01
     3       0.769577      -24.365640       5.38e-02
     4       0.779899      -24.369572       3.93e-03
     5       0.780977      -24.369601       2.97e-05
     6       0.780882      -24.369602       2.68e-07

  Converged after 6 iteration(s).

  Minimum at  x*  =  0.78088230
  f(x*)           =  -24.36960157
```

Algorithm Impllementation Results

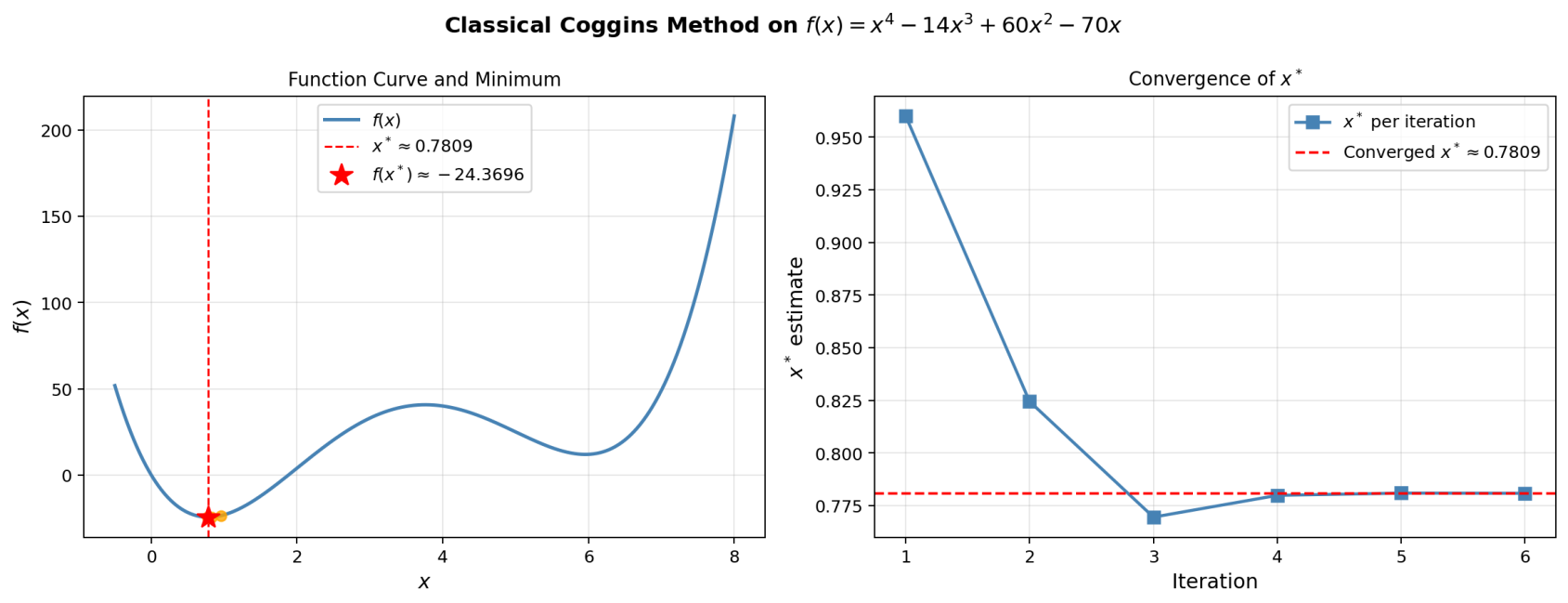

Figure 1: Left: $f(x)$ with minimum found at $x^* \approx 0.78$, $f(x^*) \approx -24.37$.
Right: convergence of x^* across iterations.

Verification

We verify $x^* \approx 0.78$ is a minimum using calculus:

First order condition:

$$f'(x) = 4x^3 - 42x^2 + 120x - 70 = 0 \quad \text{has a root near } x \approx 0.78 \checkmark$$

Second order condition:

$$f''(x) = 12x^2 - 84x + 120$$

$$f''(0.78) = 12(0.78)^2 - 84(0.78) + 120 = 61.78 > 0 \checkmark$$

Since $f''(x^*) > 0$, the curve is concave at x^*, confirming a **local minimum**.

Key Features

The classical Coggins method is efficient in that it:

- requires relatively few function evaluations.
- uses *unequally spaced* points to bracket the minimum.
- achieves rapid convergence through successive quadratic approximations.

Conclusion

Conclusion i

- Coggin's Search Method is a useful technique for step-size determination.
- It transforms multidimensional optimization into a one-dimensional search problem.
- The method improves efficiency through interpolation.

References

O. M. Bamigbola and F. B. Agusto.
Optimization on $\mathbb{R}^n$ by Coggins' method.
International Journal of Computer Mathematics, 81(9):1145–1152, 2004.

M. R. Odekunle and T. A. Badru.
Optimization on $\mathbb{R}^n$ by Coggins-Fibonacci method.
Journal of Natural Sciences, Engineering and Technology, 8(1):1–10, 2009.